Relatório

Sistemas Distribuídos - Projeto de Programação - KV Store

Nome: Anthony Deleon de Almeida Costa
RA: 11202131250

I. Link do vídeo

Link: https://youtu.be/ma8UEzSbFZc

II. Exemplos mostrados no vídeo

- GET normal (2:00): o cliente consulta uma key que o servidor já tem com timestamp igual ou maior que o do cliente. O servidor devolve o value e o timestamp dele, e o cliente imprime a linha de GET.
- GET com WAIT_FOR_RESPONSE (2:20): o cliente já viu o timestamp novo de uma key em um servidor e consulta outro servidor, que ainda tem um timestamp menor. O servidor responde WAIT_FOR_RESPONSE e fecha a conexão. Quando a replicação chega, o servidor envia o value ao cliente, sem nova requisição.
- Queda e volta de um servidor (3:10): um servidor não líder é encerrado com Ctrl+C e iniciado de novo. Ele volta sem dados, pede todas as informações ao líder e imprime "Informações recebidas do líder". Um GET nesse servidor devolve uma key gravada enquanto ele estava fora do ar.

III. Servidor

Funcionalidades

Funcionalidade	Como é tratada	Linhas
5a. Inicialização	Lê do teclado, nesta ordem, o IP e a porta do próprio servidor e o IP e a porta do líder (48-58). Compara os dois endereços: se forem iguais, este processo é o líder (60). Abre o ServerSocket na própria porta (63). Se não for o líder, inicia a thread RegistraNoLider, que o apresenta ao líder (67-69).	45-69
5b. Atendimento simultâneo	A thread principal fica em um laço com accept(). Cada conexão aceita é entregue a uma nova thread AtendeConexao (73-76), e o laço volta imediatamente a aceitar conexões. A AtendeConexao lê a Mensagem, escolhe o tratamento pelo tipo (PUT, GET, REPLICATION ou SYNC_REQUEST) (94-104) e fecha a conexão no finally, mesmo em caso de erro (106-112).	73-76 82-114
5c. PUT em servidor não líder	Imprime "Encaminhando PUT key:[key] value:[value]" (174-175). Abre uma conexão com o líder e reenvia a mesma Mensagem, que já traz o IP e a porta do cliente (177-180). Recebe o PUT_OK do líder e o repassa ao cliente pela conexão em que ele espera (182-184).	173-186
5c. PUT no líder	Imprime "Cliente [IP]:[porta] PUT key:[key] value:[value]" (188-189). Dentro de synchronized, incrementa o contador de timestamp, grava o value e o timestamp nas tabelas e copia a lista de seguidores (196-202). Como put() sobrescreve a entrada, uma key existente tem value e timestamp atualizados. Fora da trava, cria uma thread ReplicaEmSeguidor para cada seguidor (205-211), que envia a Mensagem REPLICATION com key, value e timestamp (256-265).	171-211 239-271
5d. REPLICATION	Imprime "REPLICATION key:[key] value:[value] ts:[timestamp]" (284-286). Dentro de synchronized, grava key, value e timestamp somente se o timestamp recebido for maior que o guardado, para que uma replicação atrasada nunca apague uma mais nova (288-296). Chama notifyAll() para acordar GETs em espera (299) e responde REPLICATION_OK ao líder (302-308). As linhas 276-282 são o atraso da demonstração (Seção III).	274-309
5e. REPLICATION_OK	Cada ReplicaEmSeguidor lê o REPLICATION_OK do seu seguidor na mesma conexão (266). A thread do PUT espera todas as ReplicaEmSeguidor terminarem com join() (215-220). Quando todas terminam, todos os seguidores no ar confirmaram: o líder imprime "Enviando PUT_OK ao Cliente [IP]:[porta] da key:[key] ts:[timestamp]" (222-224) e envia o PUT_OK com o timestamp e o endereço do líder (226-233). Um seguidor fora do ar faz a conexão falhar e é ignorado nesse PUT (268-269), para que o PUT não fique bloqueado.	215-233 266-269

Funcionalidade	Como é tratada	Linhas
5f. GET	Lê, dentro de synchronized, o value e o timestamp da key (320-323). Se a key existe e o timestamp do servidor é menor que o do cliente (328), imprime a linha de GET devolvendo WAIT_FOR_RESPONSE (329-332), envia a Mensagem WAIT_FOR_RESPONSE (334-341), cria a thread EsperaAtualizacao (343) e retorna; a conexão é fechada pela AtendeConexao. Se a key não existe, o value devolvido é NULL, com timestamp 0 (347-349). Nos demais casos, imprime "Cliente [IP]:[porta] GET key:[key] ts:[timestamp]. Meu ts é [ts], portanto devolvendo [value]" (351-354) e envia o value com o timestamp do servidor (356-363).	314-364
5f. Envio tardio do GET	A thread EsperaAtualizacao espera, com wait() dentro de um while, até o timestamp da key alcançar o timestamp do cliente (379-386). Ao acordar com a condição satisfeita, lê o value e o timestamp (388-389), abre uma nova conexão com a porta de escuta do cliente, informada na Mensagem do GET, e envia o value com o timestamp do servidor (392-403).	367-407
5g. Queda e volta (não líder)	Ao subir, o servidor não líder envia SYNC_REQUEST com o próprio IP e porta (123-131). Se o líder não responder, tenta de novo a cada 1 segundo, sem imprimir nada (158-165). O líder registra o seguidor e copia as duas tabelas sob a trava (419-423) e responde SYNC_RESPONSE (425-428). O seguidor mescla os dados, aceitando só timestamps maiores que os locais (134-149), chama notifyAll() (152) e imprime "Informações recebidas do líder" (155).	118-168 416-429

Estruturas de dados

Estrutura	Tipo	Para que serve / o que armazena	Linhas
Tabela de valores (tabelaValores)	`HashMap<String, String>`	Conteúdo do KV Store neste servidor. Armazena key -> value (o último value conhecido).	19-20
Tabela de timestamps (tabelaTimestamps)	`HashMap<String, Long>`	Versão de cada key, usada para comparar com o timestamp do cliente no GET e para não aceitar replicação antiga. Armazena key -> timestamp. As duas tabelas usam a mesma key e são sempre lidas e gravadas juntas, no mesmo bloco synchronized. Na prática, formam uma única tabela de hash: cada key tem um value e um timestamp. Ficaram separadas porque uma classe auxiliar "value + timestamp" seria uma quarta classe, e o enunciado permite apenas Servidor, Cliente e Mensagem.	22-23
Contador de timestamp (contadorTimestamp)	`long`	Relógio lógico do sistema, usado só pelo líder. Armazena o último timestamp gerado; cada PUT soma 1.	25-26 (uso: 197-198)
Lista de seguidores (seguidores)	`HashSet<String>`	Usada pelo líder para saber a quem enviar REPLICATION. Armazena o endereço "ip:porta" de cada seguidor registrado. Por ser um conjunto, um seguidor que volta não aparece duas vezes.	28-30 (uso: 201, 420)
Objeto de trava (trava)	`Object`	Protege as quatro estruturas acima (synchronized) e é o monitor de wait() e notifyAll() da espera do GET.	32-35
Cópia dos destinos (destinos)	`ArrayList<String>`	Cópia local da lista de seguidores, feita sob a trava no PUT e usada fora dela para replicar, sem reter a trava durante a rede.	192, 201

Estrutura	Tipo	Para que serve / o que armazena	Linhas
Threads de replicação (replicacoes)	`ArrayList<ReplicaEmSeguidor>`	Guarda as threads que enviaram REPLICATION em um PUT, para o líder esperar todas com join() antes do PUT_OK.	205-220
Identificação do servidor	`String`, `int`, `boolean`	meuIp, minhaPorta, ipLider, portaLider e souLider: endereço do próprio servidor e do líder, lidos na inicialização, e se este processo é o líder.	37-41
Mensagem (Mensagem.java)	classe	Objeto transferido em toda comunicação. Tem o tipo (constantes PUT, PUT_OK, GET, GET_RESPONSE, WAIT_FOR_RESPONSE, REPLICATION, REPLICATION_OK, SYNC_REQUEST, SYNC_RESPONSE) e atributos próprios: chave, valor, timestamp, IP e porta do cliente, IP e porta do servidor e, no SYNC_RESPONSE, cópias das duas tabelas.	Mensagem.java: 7-16, 18-39

IV. Cliente

Funcionalidades

Funcionalidade	Como é tratada	Linhas
Menu	Laço que mostra somente as opções INIT, PUT e GET e não termina por escolha do usuário (41-65). PUT e GET só são executados depois do INIT, porque antes dele o cliente não conhece nenhum servidor (54-64).	38-66
4a. INIT	Lê do teclado o IP e a porta dos três servidores e guarda cada um como "ip:porta" na lista de servidores (72-80). O cliente não pergunta nem conhece quem é o líder. Abre, uma única vez, um ServerSocket em uma porta livre (porta 0) e inicia a thread EscutaRespostas, que recebe as respostas tardias do GET (85-92).	69-95
4b. PUT	Lê do teclado a key e o value (125-129). Sorteia um dos três servidores com Random e abre a conexão (132; 224-228). Envia a Mensagem PUT com key, value, IP e porta de escuta do cliente (134-141). Recebe o PUT_OK (142-143), grava o timestamp recebido para a key (147-149) e imprime "PUT_OK key: [key] value [value] timestamp [timestamp] realizada no servidor [IP:porta]" (151-155).	124-159
4c. GET	Lê do teclado somente a key (163-164). Busca o último timestamp que o cliente tem da key, ou 0 se nunca a viu; o timestamp não vem do teclado (167-170). Sorteia um servidor (173) e envia a Mensagem GET com a key, esse timestamp, o IP e a porta de escuta (175-181). Se a resposta é WAIT_FOR_RESPONSE, imprime a linha diferenciada "GET key: [key] WAIT_FOR_RESPONSE do servidor [IP:porta], meu timestamp [ts]" e volta ao menu, sem manter a conexão (188-193). Caso contrário, imprime a resposta (195).	162-200
4c. Resposta do GET	Atualiza o timestamp da key se o do servidor for maior (206-213) e imprime "GET key: [key] value: [value] obtido do servidor [IP:porta], meu timestamp [ts do cliente] e do servidor [ts do servidor]" (215-220). É chamada tanto para a resposta imediata quanto para a tardia.	204-221
4c. Recebimento assíncrono	A thread EscutaRespostas fica em accept() na porta de escuta (110). Quando um servidor envia o value depois de um WAIT_FOR_RESPONSE, ela lê a Mensagem, fecha a conexão (111-112) e imprime a resposta, atualizando o timestamp (115).	99-121

Estruturas de dados

Estrutura	Tipo	Para que serve / o que armazena	Linhas
Lista de servidores (servidores)	`ArrayList<String>`	Permite sortear um servidor a cada PUT e GET. Armazena o "ip:porta" dos três servidores lidos no INIT.	19-20

Estrutura	Tipo	Para que serve / o que armazena	Linhas
Tabela de timestamps (meusTimestamps)	`HashMap<String, Long>`	Garante a leitura monotônica: o timestamp vai em cada GET, e o servidor nunca devolve versão anterior a ele. Armazena key -> maior timestamp já recebido. Começa vazia; key ausente vale 0.	22-23
Objeto de trava (trava)	`Object`	Protege a tabela de timestamps, usada pela thread do menu e pela EscutaRespostas.	25-27
Sorteador (sorteio)	`Random`	Escolhe o servidor aleatório de cada operação.	29
Porta de escuta (minhaPorta)	`int`	Porta escolhida pelo sistema no INIT, enviada no PUT e no GET para o servidor poder responder depois.	32-33
INIT feito (inicializado)	`boolean`	Indica se o INIT já ocorreu. Impede PUT/GET sem servidores e evita abrir a porta de escuta duas vezes.	35-36

V. Threads

Servidor

Thread	Função	Linhas
Thread principal	Lê a configuração, abre o ServerSocket e fica em accept(). Para cada conexão cria uma AtendeConexao e volta a aceitar, sem esperar o tratamento terminar.	45-79 (laço: 73-76)
AtendeConexao	Uma por conexão. Lê a Mensagem, trata PUT, GET, REPLICATION ou SYNC_REQUEST e responde. Permite atender vários clientes e servidores ao mesmo tempo.	82-114 (criação: 75)
RegistraNoLider	Só em servidor não líder. Em segundo plano, envia SYNC_REQUEST ao líder, repete a cada 1 segundo até ser atendida, mescla o estado recebido e termina.	118-168 (criação: 68)
ReplicaEmSeguidor	Criada pelo líder, uma por seguidor, em cada PUT. Envia o REPLICATION e espera o REPLICATION_OK, de modo que os seguidores são atualizados em paralelo. O líder espera todas com join().	239-271 (criação: 206-211; join: 215-220)
EsperaAtualizacao	Criada quando um GET recebe WAIT_FOR_RESPONSE. Espera com wait() até a key chegar à versão do cliente e então envia a resposta à porta de escuta do cliente. O wait() libera a trava, então o servidor continua atendendo durante a espera.	367-407 (criação: 343)

Cliente

Thread	Função	Linhas
Thread principal	Mostra o menu, lê o teclado e envia PUT e GET, esperando a resposta imediata de cada um.	38-66
EscutaRespostas	Criada no INIT. Fica em accept() na porta de escuta e recebe as respostas tardias do GET (depois de um WAIT_FOR_RESPONSE). Por causa dela, o menu continua disponível e o cliente não mantém a conexão aberta durante a espera.	99-121 (criação: 89)

VI. Proteção de acessos concorrentes

No servidor, várias threads acessam as mesmas estruturas ao mesmo tempo: PUTs concorrentes de keys diferentes no líder, GETs, REPLICATIONs, as threads de espera e a recuperação de um seguidor. A proteção

é feita com blocos synchronized sobre um único objeto de trava (Servidor.java, linha 35), de modo que só uma thread por vez lê ou altera as tabelas, o contador e a lista de seguidores.
Nenhum bloco synchronized faz operação de rede: o PUT copia a lista de seguidores dentro da trava e replica fora dela, e o GET lê value e timestamp juntos e responde depois. Assim, um seguidor lento não bloqueia os outros pedidos. A espera do WAIT_FOR_RESPONSE usa wait() e notifyAll() no mesmo objeto de trava; o wait() fica dentro de um while porque notifyAll() acorda todas as threads de espera, inclusive as de outras keys.

Ponto protegido	**O que protege**	**Linhas**
PUT no líder	Incremento do contador, gravação nas duas tabelas e cópia da lista de seguidores, como uma operação única.	Servidor 196-202
REPLICATION	Comparação de timestamp e gravação nas tabelas; notifyAll() para acordar GETs em espera.	Servidor 288-300 (notifyAll: 299)
GET	Leitura do value e do timestamp da mesma versão.	Servidor 320-323
Espera do GET	while + wait() até a versão pedida chegar, e leitura do value e do timestamp.	Servidor 379-390 (wait: 382)
Recuperação (seguidor)	Mescla do estado recebido do líder; notifyAll().	Servidor 134-153 (notifyAll: 152)
Recuperação (líder)	Registro do seguidor e cópia das duas tabelas, para não enviar tabela pela metade.	Servidor 419-423
Leitura auxiliar	timestampDaChave lê a tabela de timestamps e só é chamada dentro dos blocos acima.	Servidor 410-413
Cliente	Tabela de timestamps, usada pela thread do menu e pela EscutaRespostas.	Cliente 27; 147-149; 167-170; 206-213

VII. Links das fontes do código

- https://www.tutorialspoint.com/java/java_multithreading.htm
- https://github.com/google/gson
- http://tutorials.jenkov.com/java-json/gson.html